PROJECT REPORT

IoT-BASED REAL-TIME FLOOD MONITORING AND EARLY WARNING SYSTEM

FOSSEE Open Source Hardware Make-A-Thon 2026

RCC Institute of Information Technology, Kolkata

Submitted by
Siddhartha Banerjee

Sreejit Mondal

Prabha Bhattacharya

Abhik Saha

Rupam Mondal

ABSTRACT

Flooding can develop from rapidly rising water, stormwater accumulation, drainage overflow, or other local water-accumulation mechanisms. A monitoring system therefore needs to observe the water level continuously and communicate the trend to a remote operator. This project presents a single-station IoT-based flood monitoring and early-warning prototype built around a NodeMCU ESP8266. A JSN-SR04T waterproof ultrasonic sensor provides non-contact water-level measurements, while a BME280 measures ambient temperature, humidity, and pressure. The ESP8266 filters the ultrasonic readings, estimates water depth relative to a calibrated reference distance, computes the rate of rise, and assigns a flood status using predefined depth thresholds. Local indication is provided through a 0.96-inch I2C OLED, green/yellow/red LEDs, and a buzzer. The device transmits telemetry to ThingsBoard Cloud over Wi-Fi using MQTT over TLS. The dashboard displays current water depth, flood status, rate of rise, environmental measurements, and a real-time water-depth trend graph.

The final prototype was experimentally validated in a controlled container-based test. Representative observations included 0.00 cm for the empty condition, approximately 2.11-2.54 cm for the Moderate range, 4.60-4.99 cm for the High range, and 6.24-6.72 cm for the Critical range. The corresponding ThingsBoard dashboard displayed the same live values and status transitions. The testing demonstrates the feasibility of an open-source single-node prototype for real-time flood-level monitoring and remote visualization. The present design is intentionally limited to water-level and rate-of-rise monitoring; it does not determine the physical cause of flooding and does not yet implement multi-node LoRa communication, rainfall sensing, flow sensing, or solar power.

Keywords: Flood monitoring, Internet of Things (IoT), ESP8266, JSN-SR04T, BME280, MQTT, ThingsBoard, water level, rate of rise, early warning.

TABLE OF CONTENTS

1 Introduction
2 Problem Statement
3 Objectives
4 Proposed Solution and Scope
5 System Architecture
6 Hardware Components
7 Software and IoT Communication Architecture
8 Circuit Design and Pin Mapping
9 Methodology and Algorithms
10 Implementation
11 Testing and Experimental Results
12 Advantages
13 Limitations
14 Future Scope
15 Conclusion

1 Introduction

Flooding is a highly dynamic phenomenon in which water levels can change over time and may quickly exceed a safe local condition. Conventional observation methods often depend on visual inspection or periodic manual measurement. Such approaches provide limited temporal resolution and can delay awareness when water starts to rise rapidly. Internet of Things (IoT) technology provides a practical way to combine local sensing, edge processing, wireless communication, and cloud visualization in a single monitoring system.

The present project develops a single-station prototype for continuous local water-level monitoring. The system uses a waterproof ultrasonic probe so that the sensing element can be placed above the water surface without direct electrical contact with the water. A NodeMCU ESP8266 acts as the edge controller and performs the sensor acquisition and decision logic. A BME280 adds environmental telemetry for context. The processed information is displayed locally and transmitted to ThingsBoard Cloud using MQTT over TLS, where it can be viewed remotely through a web dashboard.

The report first defines the problem, objectives, and scope, then presents the system architecture and hardware and software implementation. It next describes the circuit, algorithms, and experimental validation, and closes with the project's advantages, limitations, future work, conclusion, and references.

1.1 Motivation

The project demonstrates how a compact embedded controller can transform distance measurements into a decision-oriented flood indicator and make that information available to a remote operator. The report documents the hardware, wiring, software behavior, and test evidence so that the prototype can be understood and extended.

1.2 Project Scope

- The prototype monitors water depth at one physical sensing station.
- The system calculates a rate-of-rise indicator from successive depth measurements.
- Flood status is assigned from predefined water-depth thresholds.
- Environmental telemetry includes temperature, humidity, and pressure.
- Local indication uses an OLED display, three LEDs, and a buzzer.
- Cloud telemetry is transmitted over Wi-Fi using MQTT to ThingsBoard Cloud.
- The current implementation intentionally excludes a rain gauge, flow sensor, LoRa multi-node network, solar power, and automated flood-cause classification.

2 Problem Statement

A flood-monitoring prototype should answer three practical questions continuously: (1) how much water is present relative to a known reference level, (2) whether the water level is rising or falling and how quickly, and (3) what qualitative flood state should be communicated to an operator. The system should also remain useful when the operator is not physically beside the sensor. The engineering problem is therefore to acquire stable water-level data, derive meaningful indicators, communicate them over a network, and present them in a simple human-readable interface.

The prototype addresses this problem using non-contact ultrasonic measurement, filtering and threshold logic on the ESP8266, and cloud visualization. It is a monitoring and early-warning demonstrator rather than a complete hydrological forecasting system. In particular, the prototype cannot identify whether the water increase was caused by heavy rainfall, drainage blockage, river overflow, or another physical mechanism without additional sensing inputs.

3 Objectives

The main objectives of the project are summarized below.

No.	Objective
1	Measure water-surface distance using a waterproof ultrasonic sensor.
2	Convert the measured distance into water depth relative to a calibrated reference distance.
3	Filter short-term measurement variation to obtain a more stable depth value.
4	Calculate the rate of water-level change and identify stable, rising, or falling conditions.
5	Classify the observed water depth into Normal, Moderate, High, and Critical states.
6	Provide local indication using an OLED display, LEDs, and a buzzer.
7	Measure temperature, humidity, and pressure using a BME280.
8	Transmit telemetry to ThingsBoard Cloud using Wi-Fi and MQTT over TLS.
9	Provide a web dashboard containing live values and water-depth history.
10	Document the prototype hardware, software behavior, experimental procedure, and observed limitations.

4 Proposed Solution and Scope

The proposed system combines an edge-computing sensor node with a cloud dashboard. The JSN-SR04T provides water-surface distance; the BME280 provides ambient environmental measurements. The NodeMCU ESP8266 performs calibration, filtering, water-depth calculation, rise-rate estimation, status classification, local display/alerting, and MQTT communication. ThingsBoard Cloud receives the telemetry and provides the dashboard for remote monitoring.

A single-node design was selected for the present prototype so that the complete end-to-end path could be built and verified with real hardware. Multi-node deployment, long-range communication, additional hydrological sensors, and renewable power are treated as future extensions rather than components of the final validated prototype.

4.1 Design Principles

- Non-contact water measurement to keep the electrical sensing circuitry away from the water surface.
- Edge processing so that status decisions remain available locally even when cloud communication is temporarily unavailable.
- Dual indication: local alarm/display plus remote dashboard.
- Use of open and widely available embedded hardware and software tools.
- Modular architecture so that rainfall, flow, additional water-level nodes, or alternative communication links can be added later.

5 System Architecture

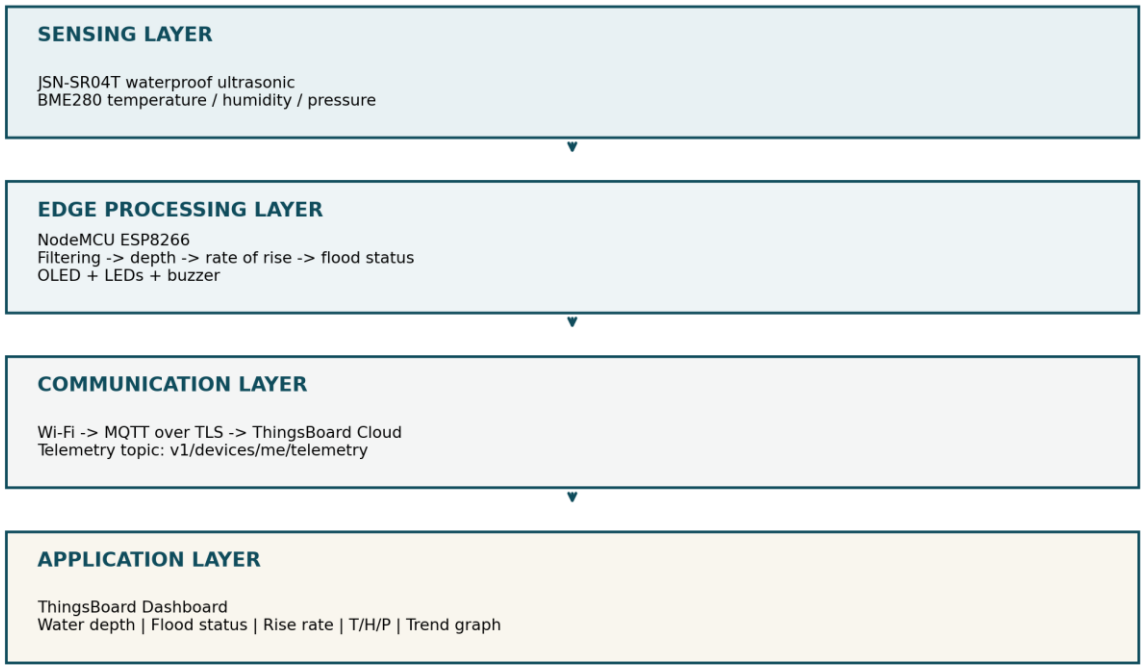


Figure 1. Overall system architecture

5.1 Sensing Layer

The sensing layer contains the JSN-SR04T waterproof ultrasonic sensor and the BME280 environmental sensor. The ultrasonic sensor provides the primary water-level measurement. The BME280 operates over I2C and was verified during testing at address 0x76, with SDO connected to ground and CSB connected to 3.3 V.

5.2 Edge Processing Layer

The NodeMCU ESP8266 receives the sensor data and performs the main embedded logic. A calibration stage establishes the reference distance. The ultrasonic signal is sampled periodically, processed using median and moving-average filtering, converted to depth, and used to calculate the rate of rise. The current firmware uses depth-based thresholds for the flood-state classification. Local indication is handled through the OLED, LEDs, and buzzer.

5.3 Communication Layer

The node connects to a Wi-Fi network and publishes telemetry to ThingsBoard Cloud over MQTT. The final prototype uses the standard ThingsBoard telemetry topic `v1/devices/me/telemetry`. ThingsBoard documents this topic as the standard device telemetry topic, and its Cloud MQTT service supports TLS on port 8883. [1][2]

5.4 Application Layer

The ThingsBoard dashboard is the application layer. It displays the latest water depth, flood status, rate of rise, temperature, humidity, and pressure. A time-series chart records water-depth history over a rolling 30-minute window in the configured dashboard.

5.5 Overall Workflow

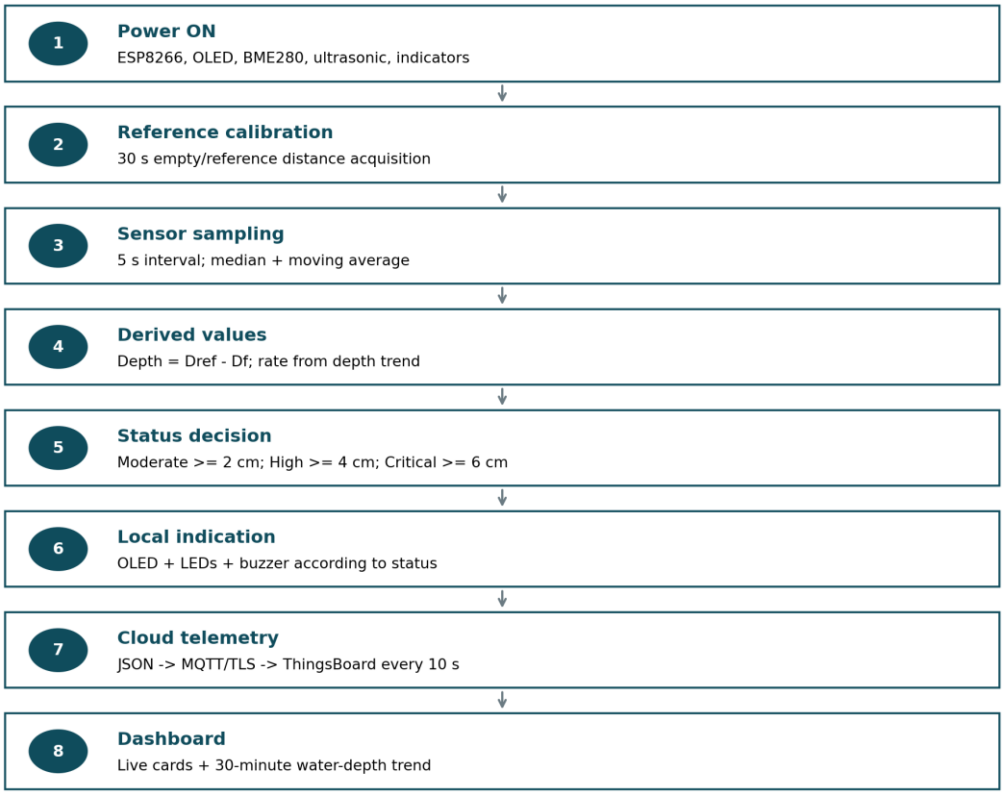


Figure 2. End-to-end system workflow

6 Hardware Components

The implemented prototype combines a NodeMCU ESP8266 controller, a waterproof ultrasonic water-level sensor, an environmental sensor, and local display and warning devices. The summary table below identifies each item; the following descriptions document its purpose, actual connections, and role in the assembled system.

Component	Role in the prototype	Implementation detail
NodeMCU ESP8266 ESP-12E	Central controller and Wi-Fi connectivity	Runs Arduino firmware; handles sensors, indicators, filtering, status logic, and MQTT.
JSN-SR04T waterproof ultrasonic sensor	Non-contact water-surface distance measurement	Trigger/Echo interface; ECHO uses a resistor divider before D6.
BME280	Ambient temperature, humidity, and pressure	I2C; SCL=D1, SDA=D2; SDO=GND; CSB=3V3; observed address 0x76.
0.96-inch I2C OLED	Local numerical/status display	I2C at 0x3C; displays depth, rise rate, and system state.
Green / Yellow / Red LEDs	Visual status indication	Each LED uses a 220-ohm series resistor.

Component	Role in the prototype	Implementation detail
Buzzer	Audible warning	Wiring plan connects + to D7 and - to GND; verify current rating before direct GPIO drive.
1 kOhm + 2 kOhm resistors	Ultrasonic ECHO level shifting	Divider reduces approximately 5 V ECHO to about 3.3 V at D6.
Breadboard and jumper wires	Prototype interconnection	Common ground shared by all modules.
USB power / cable	Primary prototype power source	USB powers NodeMCU; 3V3 supplies BME280/OLED; use VIN/5V for JSN only if this board exposes USB 5 V there.

6.1 NodeMCU ESP8266 ESP-12E Controller

The NodeMCU ESP8266 ESP-12E is the central controller and Wi-Fi interface. Its firmware samples the sensors, filters the ultrasonic readings, calculates water depth and rate of rise, classifies the configured flood state, updates the local indicators, and publishes telemetry to ThingsBoard. The board is powered and programmed through USB. In this prototype, D5/GPIO14 and D6/GPIO12 serve the ultrasonic TRIG and ECHO signals; D1/GPIO5 and D2/GPIO4 form the shared I2C bus; D0, D3, and D4 drive the LEDs; and D7 drives the buzzer. The VIN/5V pin supplies the JSN-SR04T module.

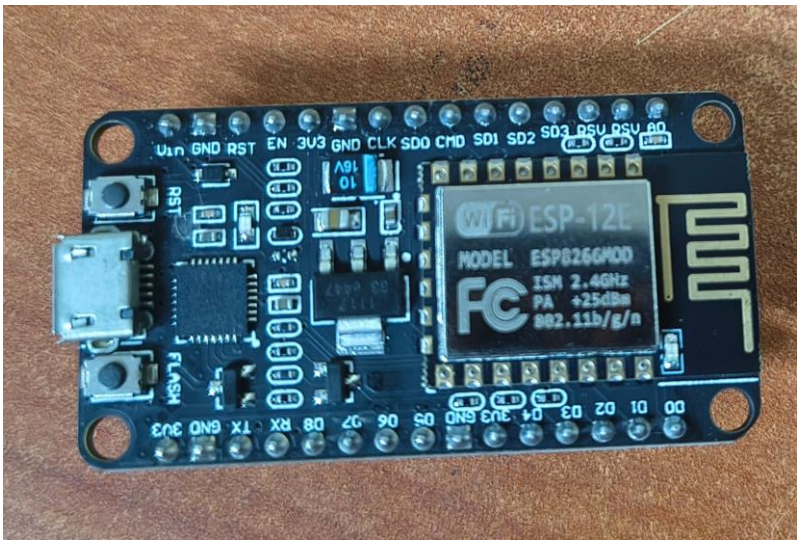


Figure 3. NodeMCU ESP8266 ESP-12E development board used as the prototype controller.

6.2 JSN-SR04T Waterproof Ultrasonic Sensor

The JSN-SR04T measures distance to the water surface without placing its electronics in contact with the water. Its separate waterproof probe is positioned above and aligned with the reflecting surface; the controller measures the interval between the trigger pulse and returned echo. The module is powered from NodeMCU VIN/5V and shares the system ground. TRIG connects to D5/GPIO14. ECHO reaches D6/GPIO12 through a resistor divider: connect 1 kΩ from ECHO to the D6 junction, then 2 kΩ from that junction to GND. With a 5 V ECHO signal this ratio produces approximately 3.3 V at D6. Confirm the resistor orientation and common ground on the physical prototype before powering it; do not connect a 5 V ECHO directly to an ESP8266 input.

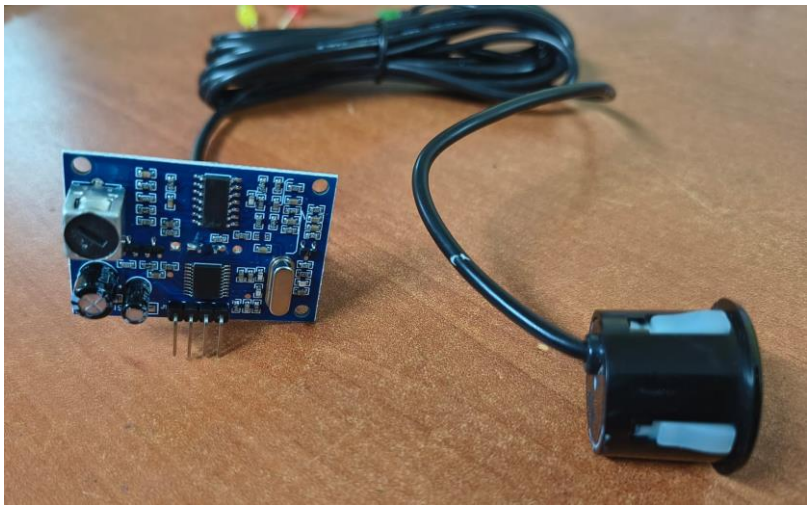


Figure 4. JSN-SR04T controller board and its wired waterproof ultrasonic probe.

6.3 BME280 Environmental Sensor

The BME280 adds ambient temperature, relative humidity, and atmospheric pressure to the reported telemetry. It communicates in I2C mode from the 3.3 V rail and shares the bus with the OLED. SCL connects to D1/GPIO5 and SDA to D2/GPIO4; CSB is tied to 3.3 V to select I2C, while SDO is tied to GND. This configuration gives I2C address 0x76, consistent with the sensor's address selection [4]. The BME280 provides environmental context and is not used to measure water level.

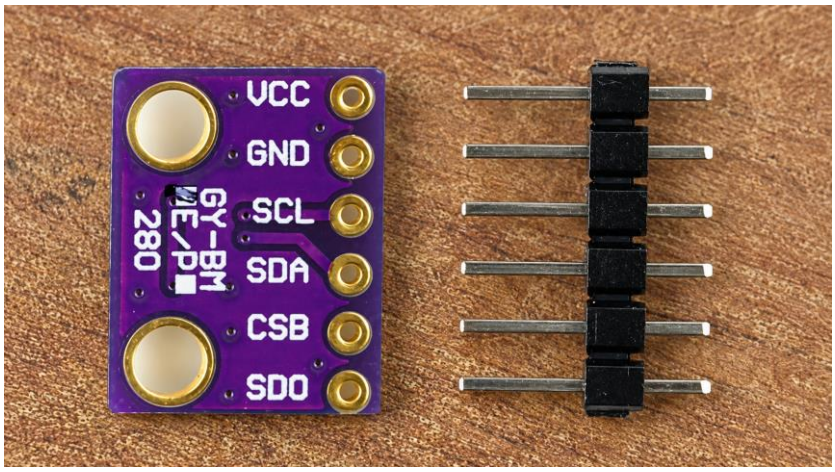


Figure 5. Six-pin BME280 breakout board used for environmental measurements.

6.4 0.96-Inch I2C OLED Display

The 0.96-inch OLED provides a local view of the calculated water depth, rate of rise, and flood status. It is powered from 3.3 V and GND and uses the shared I2C bus: SCL connects to D1/GPIO5 and SDA to D2/GPIO4. Sharing the two signal lines with the BME280 reduces the number of controller pins needed while allowing both devices to operate on the same bus.



Figure 6. 0.96-inch I2C OLED module used for local system readings.

6.5 Green, Yellow, and Red Status LEDs

Three LEDs provide at-a-glance visual status indication. Each LED has its own 220 Ω series resistor. The green, yellow, and red LED control connections are D0/GPIO16, D3/GPIO0, and D4/GPIO2 respectively; their returns connect to the common ground. The firmware selects the indicators to reflect the system's flood-status state. D3/GPIO0 and D4/GPIO2 also participate in ESP8266 boot-mode selection, so the LED circuits must preserve the required levels during reset [8]. The LEDs are visible in the complete breadboard assembly photograph below.

6.6 Two-Pin Piezo Buzzer

The two-pin piezo buzzer provides an audible local warning. The connection diagram shows its positive lead connected to D7/GPIO13 and its negative lead to GND. Before powering it directly from the GPIO, check the buzzer's rated current against the ESP8266 output capability; the photograph does not establish its current requirement. Use a transistor driver if the buzzer requires more current than the GPIO can safely provide.

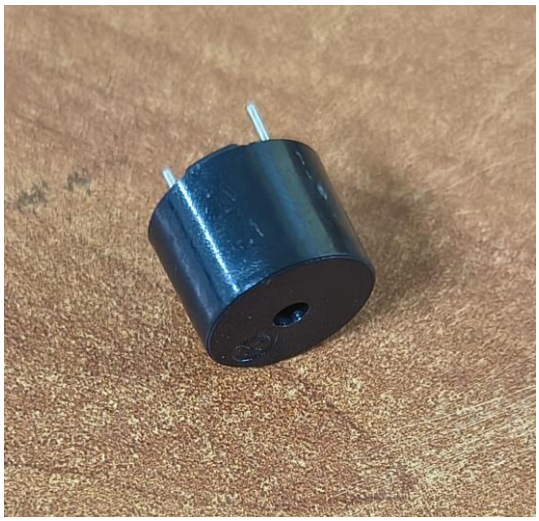


Figure 7. Two-pin piezo buzzer used for audible warning output.

6.7 Breadboard, Jumper Wires, and Power

The breadboards and jumper wires provide the prototype's temporary electrical and mechanical interconnections. Connect NodeMCU GND to the breadboard ground rail and NodeMCU 3V3 to the 3.3 V rail used by the BME280 and OLED; do not place 5 V on that rail. USB powers the NodeMCU. The wiring plan powers the JSN-SR04T from VIN/5V, provided the specific NodeMCU board exposes USB 5 V on that pin. Verify this board connection before powering the ultrasonic module. All components share a common ground. In the ultrasonic ECHO divider, place 1 kΩ between ECHO and D6 and 2 kΩ between D6 and GND.

6.8 Complete Prototype Assembly

The assembled breadboard prototype brings the controller, sensor interfaces, local display, LEDs, buzzer, and resistor divider together as one working monitoring node. The photograph records the overall hardware circuit and wiring arrangement used for the laboratory prototype.

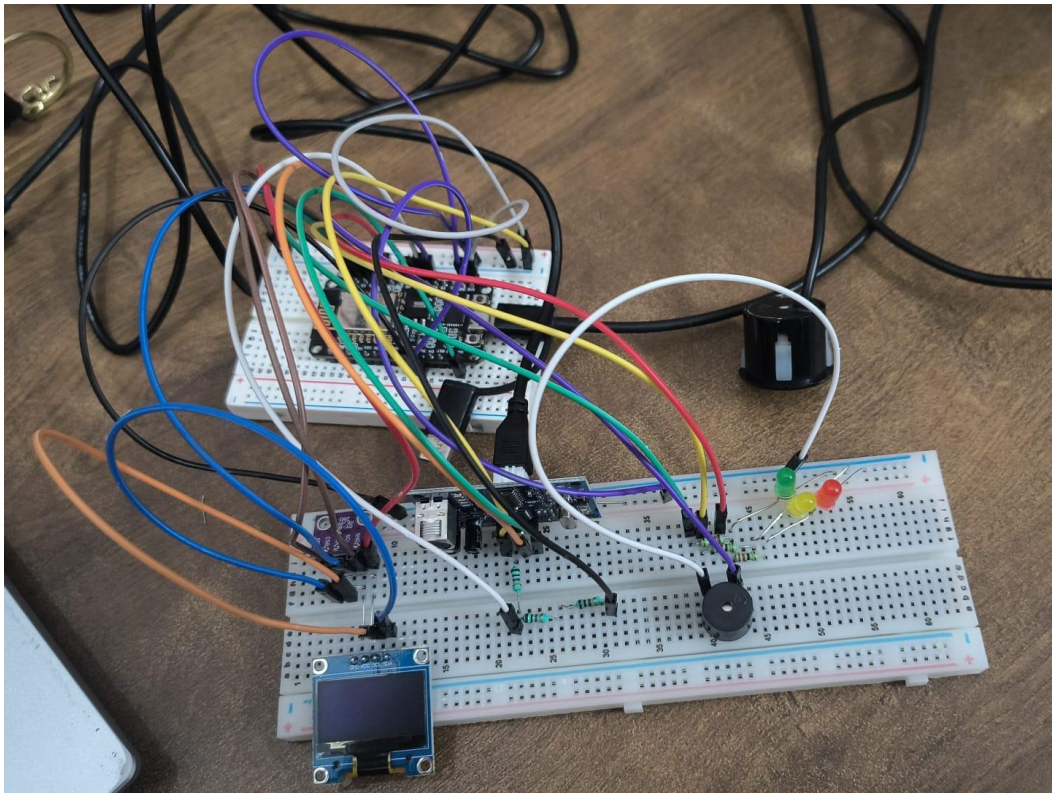


Figure 8. Overall hardware circuit and assembled breadboard prototype.

6.9 Hardware Operation and Wiring Notes

- The JSN-SR04T probe is mounted above the target water surface and kept aligned with the reflecting surface.
- For the ultrasonic ECHO divider, connect 1 kΩ from ECHO to the D6/GPIO12 junction and 2 kΩ from that junction to GND; verify this orientation before applying power.
- The BME280 and OLED share the 3.3 V I2C bus on D1/GPIO5 (SCL) and D2/GPIO4 (SDA).
- Connect NodeMCU GND to the common ground rail and 3V3 to the BME280/OLED rail; keep 5 V off the 3.3 V rail. Confirm that this board exposes USB 5 V at VIN before powering the JSN-SR04T.
- The open breadboard assembly is intended for controlled laboratory demonstration, not permanent outdoor deployment.
- Before connecting the buzzer directly to D7/GPIO13, verify its rated current; use a transistor driver if the GPIO cannot safely supply it.

7 Software and IoT Communication Architecture

Software / Library	Purpose
Arduino IDE	Firmware development, compilation, serial monitoring, and upload.
ESP8266 Arduino core / ESP8266WiFi	Wi-Fi connection and ESP8266 platform support.
Wire	I2C communication with OLED and BME280.
Adafruit_GFX + Adafruit_SSD1306	OLED rendering.
Adafruit_BME280	BME280 initialization and environmental data acquisition.
PubSubClient	MQTT client for publishing telemetry.
WiFiClientSecure	TLS-capable network client for the ThingsBoard Cloud MQTT connection.
ThingsBoard Cloud	Cloud telemetry storage and web dashboard.
Fritzing	Physical/breadboard wiring documentation.

7.1 MQTT Telemetry

The device publishes a JSON object containing the current derived values and BME280 measurements. ThingsBoard records telemetry key-value pairs as time-series data that can be used by dashboard widgets. [1]

Key	Meaning	Example
water_depth_cm	Filtered water depth	6.24
rate_of_rise_cm_min	Water-level rate of change	3.46
temperature_C	Ambient temperature	31.60
humidity_percent	Relative humidity	69.54
pressure_hPa	Atmospheric pressure	1009.20
flood_status	Text status	CRITICAL
flood_status_code	Numeric status code	3

7.2 Security Consideration

The final prototype uses an encrypted MQTT transport to the ThingsBoard Cloud endpoint on port 8883. During development, the ESP8266 connection used certificate-validation bypass (`setInsecure()`) because a CA certificate was not yet provisioned. This is acceptable as a temporary laboratory development setting but should be replaced by normal CA certificate validation before field deployment. ThingsBoard Cloud documents TLS on port 8883 for MQTT connections. [2]

8 Circuit Design and Pin Mapping

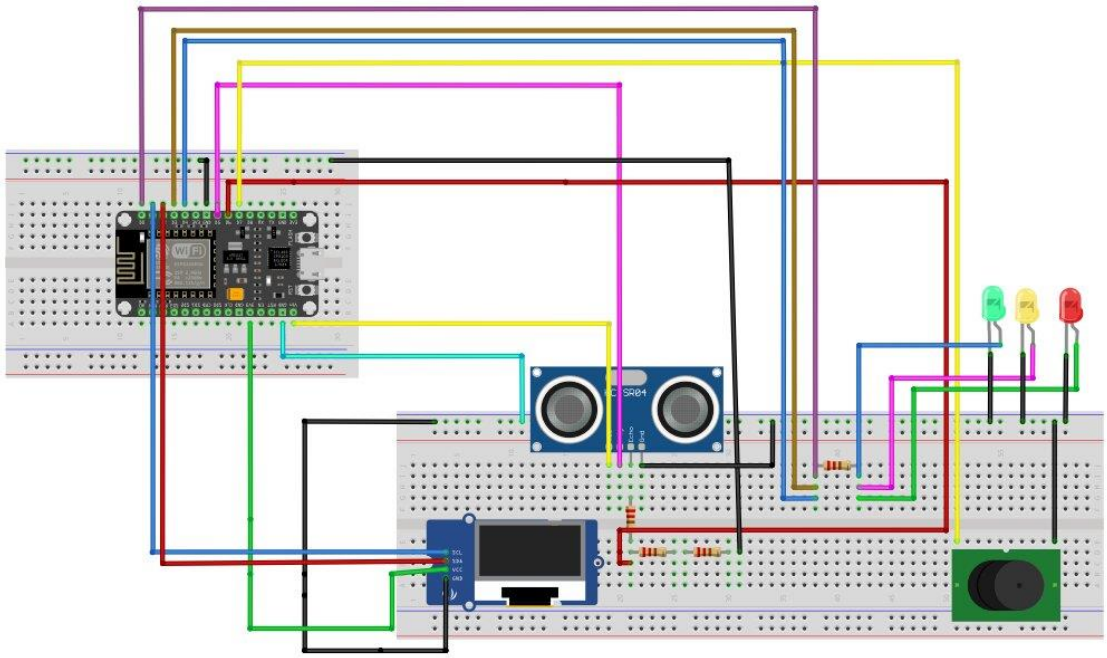


Figure 9. Simplified hardware interconnection summary

The wiring drawing uses a generic two-transducer ultrasonic symbol; the physical prototype uses the JSN-SR04T controller board and its separate waterproof probe shown in Figure 4.

Device	Signal / Pin	NodeMCU connection	Notes
JSN-SR04T	VCC	VIN / 5 V	USB-powered NodeMCU rail
JSN-SR04T	GND	GND	Common ground
JSN-SR04T	TRIG	D5 / GPIO14	Digital output from ESP8266
JSN-SR04T	ECHO	D6 / GPIO12	Through 1 kOhm / 2 kOhm divider
OLED	VCC	3V3	Do not apply 5 V to 3V3 rail
OLED	GND	GND	Common ground
OLED	SCL	D1 / GPIO5	I2C clock
OLED	SDA	D2 / GPIO4	I2C data
BME280	VCC	3V3	I2C mode
BME280	GND	GND	Common ground
BME280	SCL	D1 / GPIO5	Shared I2C bus
BME280	SDA	D2 / GPIO4	Shared I2C bus

Device	Signal / Pin	NodeMCU connection	Notes
BME280	CSB	3V3	Selects I2C mode
BME280	SDO	GND	Address observed as 0x76
Green LED	Anode -> 220 Ohm	D0 / GPIO16	Cathode -> GND
Yellow LED	Anode -> 220 Ohm	D3 / GPIO0	Cathode -> GND
Red LED	Anode -> 220 Ohm	D4 / GPIO2	Cathode -> GND
Buzzer	+	D7 / GPIO13	Check rated current before direct GPIO drive; add a transistor driver if needed.
Buzzer	-	GND	Common ground

The detailed Fritzing breadboard diagram is treated as a separate hardware-design deliverable. The pin map above is included in this report so the wiring can be reproduced without relying on an image alone.

9 Methodology and Algorithms

9.1 Ultrasonic Distance Measurement

The JSN-SR04T operates by sending an ultrasonic pulse and measuring the duration of the returned echo. The embedded program converts the echo time into distance using the usual ultrasonic time-of-flight relationship. In this prototype, the raw reading is not used directly for status classification; it is filtered first.

9.2 Calibration

At startup, the prototype performs a 30-second reference-calibration interval. The reference distance represents the sensor-to-reference surface distance under the current mounting arrangement. During the final validation run the calibrated reference distance was approximately 65.87 cm. The system therefore reports water depth relative to this local reference instead of requiring the user to enter the physical tank height manually.

For a deployed version, the calibration routine should be modified so that an already-flooded installation cannot be accidentally calibrated as the zero-water condition.

9.3 Filtering

Two stages of smoothing are applied to reduce short-term ultrasonic variation: a median filter is used to reject isolated outliers, followed by a moving-average stage that further smooths the measurement before water-depth calculation. The current prototype uses seven samples for the median stage and five samples for the moving-average stage.

9.4 Water-Depth Calculation

Water depth (cm) = Reference distance (cm) - Filtered distance (cm)

Negative values are constrained to zero in the displayed depth so that an empty reference condition is reported as approximately 0 cm.

9.5 Rate-of-Rise Calculation

Rate of rise (cm/min) = (current depth (cm) - previous depth (cm)) / elapsed time (s) × 60

A positive value indicates a rising level, a negative value a falling level, and a near-zero value a stable level. Changes smaller than 0.15 cm are treated as zero, and the reported rate is clipped to the range -5.00 to +5.00 cm/min. Rate of rise is a trend metric; the present flood-status classification is based primarily on depth.

9.6 Flood-Status Classification

Status	Depth condition	Prototype meaning
NORMAL	< 2 cm	No significant water accumulation relative to the local reference.
MODERATE	2 cm to < 4 cm	Water accumulation has crossed the first warning band.
HIGH	4 cm to < 6 cm	Higher local water level requiring increased attention.
CRITICAL	>= 6 cm	Highest prototype alert state; depth has crossed the critical threshold.

9.7 Local Indication Logic

The local interface mirrors the computed state so that the prototype remains useful even when the cloud dashboard is not being viewed. The OLED provides numerical depth and rise-rate information, while the three LEDs provide a simple status color and the buzzer supplies an audible warning condition.

10 Implementation

The firmware integrates sensor acquisition, filtering, local display, Wi-Fi connectivity, MQTT publishing, and status indication into one main loop. A typical cycle is: acquire ultrasonic data, update filtered distance, calculate water depth and rate of rise, read BME280 values, determine status, update local indicators, and publish telemetry when the telemetry interval expires. The ESP8266 Arduino core, PubSubClient, and Adafruit display and sensor libraries provide platform and peripheral support [5][6][7].

10.1 Timing Parameters

Parameter	Prototype value	Purpose
Calibration interval	30 s	Establish initial reference distance
Sensor interval	5 s	Acquire and process water-level samples
MQTT telemetry interval	10 s	Publish latest telemetry to ThingsBoard
Serial Monitor	115200 baud	Development-time observation

10.2 ThingsBoard Dashboard

The dashboard contains the following widgets: Water Depth value card, Flood Status value card with status-dependent color, Rate of Rise HTML value card, Temperature value card, Humidity value card,

Pressure value card, and a Water Depth vs Time time-series chart. This arrangement was selected to keep the most decision-relevant values in the top row while maintaining historical context below.

11 Testing and Experimental Results

Testing was performed with a controlled water container and the sensor mounted above the water surface. The objective was to verify the end-to-end behavior rather than to claim metrological accuracy against a laboratory reference instrument. Tests included the empty condition, progressive water addition through the threshold ranges, stable water levels after filling, and return to the empty condition.

11.1 Final Validation Dataset

Test state	Representative observed depth	Representative rise rate	Status	Observation
Empty	0.00 cm	0.00 cm/min	NORMAL	Stable zero-water condition.
Moderate	2.11-2.54 cm	1.85-3.38 cm/min	MODERATE	Depth crossed the 2 cm threshold.
High	4.60-4.99 cm	0.00-5.00 cm/min	HIGH	Depth remained between 4 and 6 cm.
Critical	6.24-6.72 cm	-5.00 to 5.00 cm/min	CRITICAL	Depth exceeded the 6 cm critical threshold.
Stable after filling	2.94-2.95 cm	0.00 cm/min	MODERATE	Rate settled to zero when water level stopped changing.

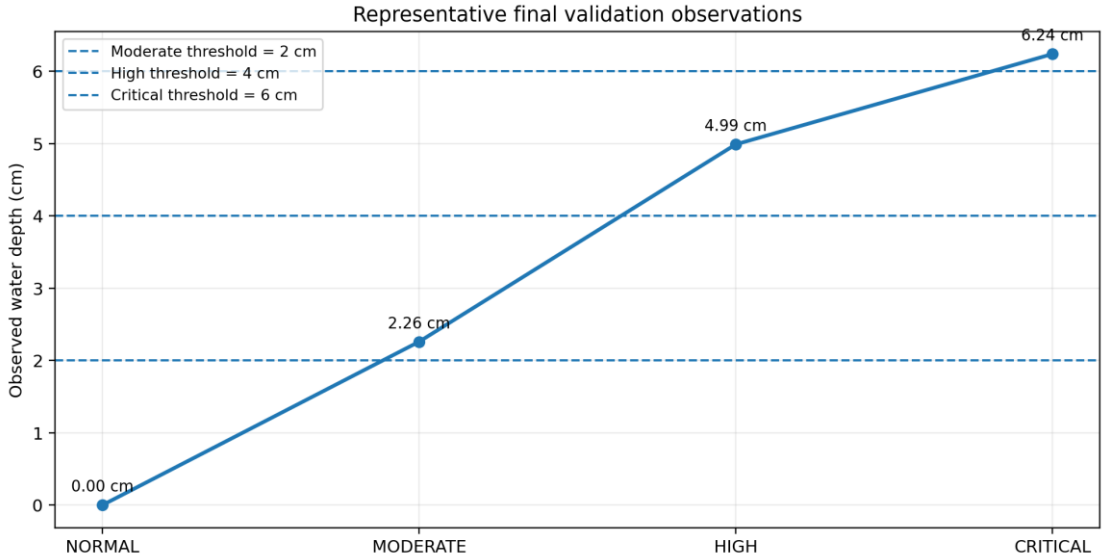


Figure 10. Representative final validation observations used in the report

The test sequence demonstrates the intended state transitions. At the empty condition, the system reports 0.00 cm and 0.00 cm/min. When water was added, the depth entered the Moderate band at approximately 2.11-2.54 cm. Continued filling produced High-state values around 4.60-4.99 cm. Further filling produced Critical values of 6.24-6.72 cm. When the water level became stable, the rate-of-rise returned to 0.00 cm/min.

11.2 Environmental Telemetry Observations

Measurement	Observed during final tests
Temperature	Approximately 31.5-31.7 C
Humidity	Approximately 69.3-70.1 %
Pressure	Approximately 1009.2-1009.6 hPa

These environmental values were recorded to verify that the BME280 was being sampled and transmitted alongside the water-level variables. They are included as contextual laboratory observations rather than as a claim of calibrated environmental accuracy.

11.3 Dashboard Validation

The following screenshots provide direct visual evidence that the cloud dashboard followed the locally calculated states. The displayed dashboard values match the corresponding serial telemetry for the representative test states.

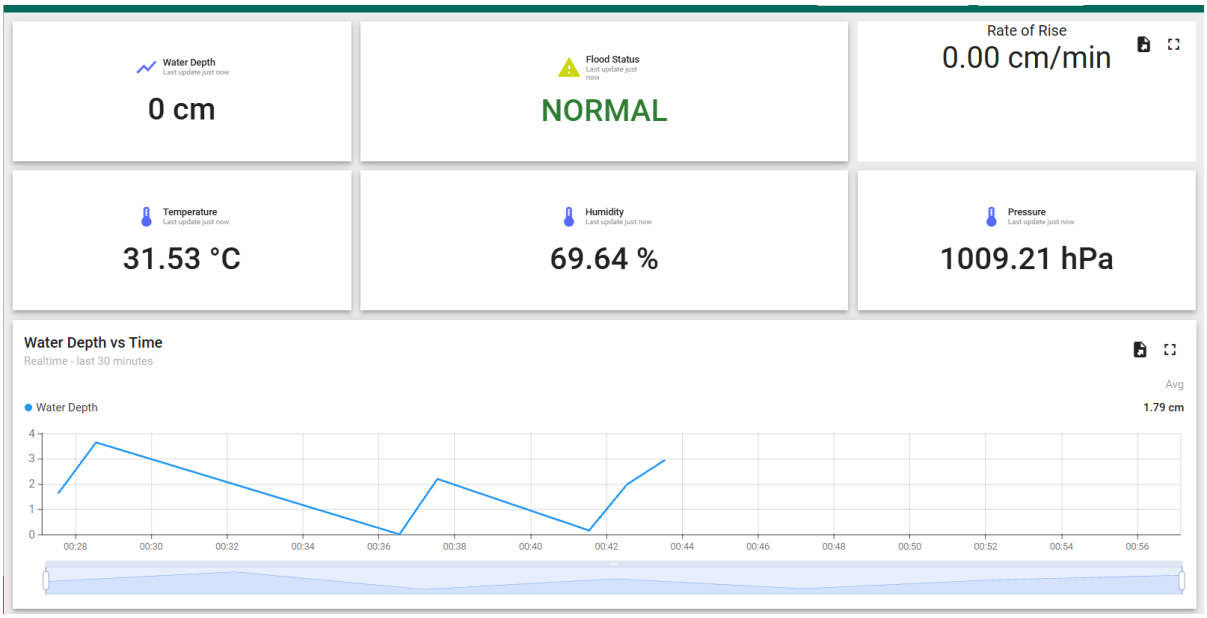


Figure 11. ThingsBoard dashboard in NORMAL state: 0 cm water depth and 0.00 cm/min rise rate

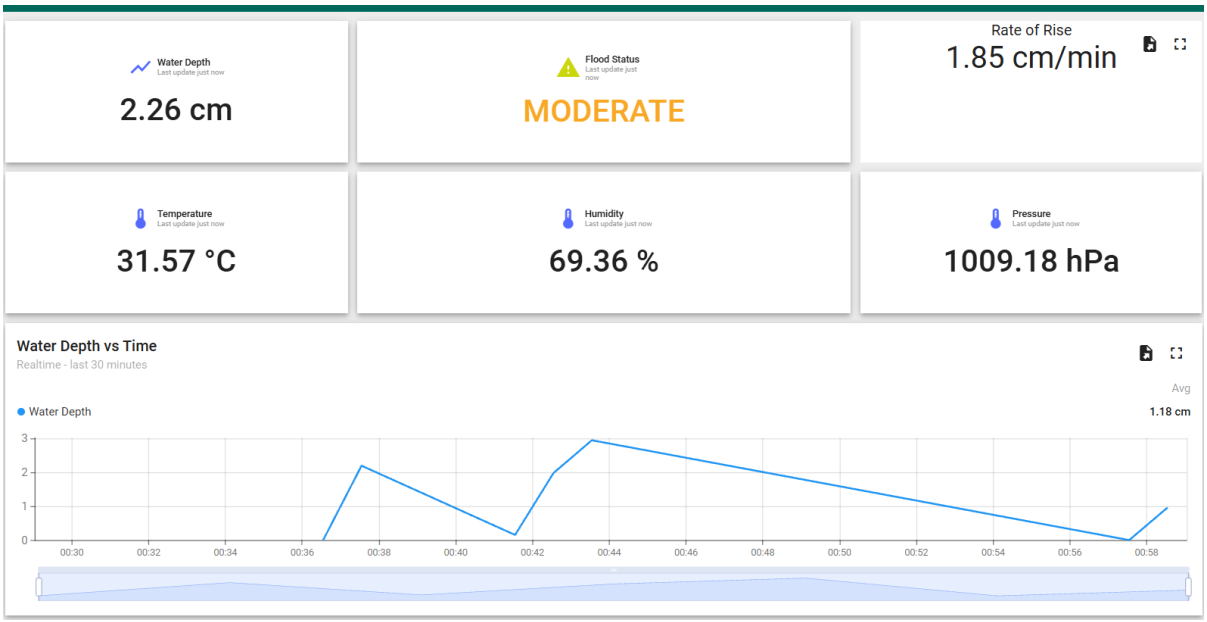


Figure 12. ThingsBoard dashboard in MODERATE state: 2.26 cm water depth and 1.85 cm/min rise rate

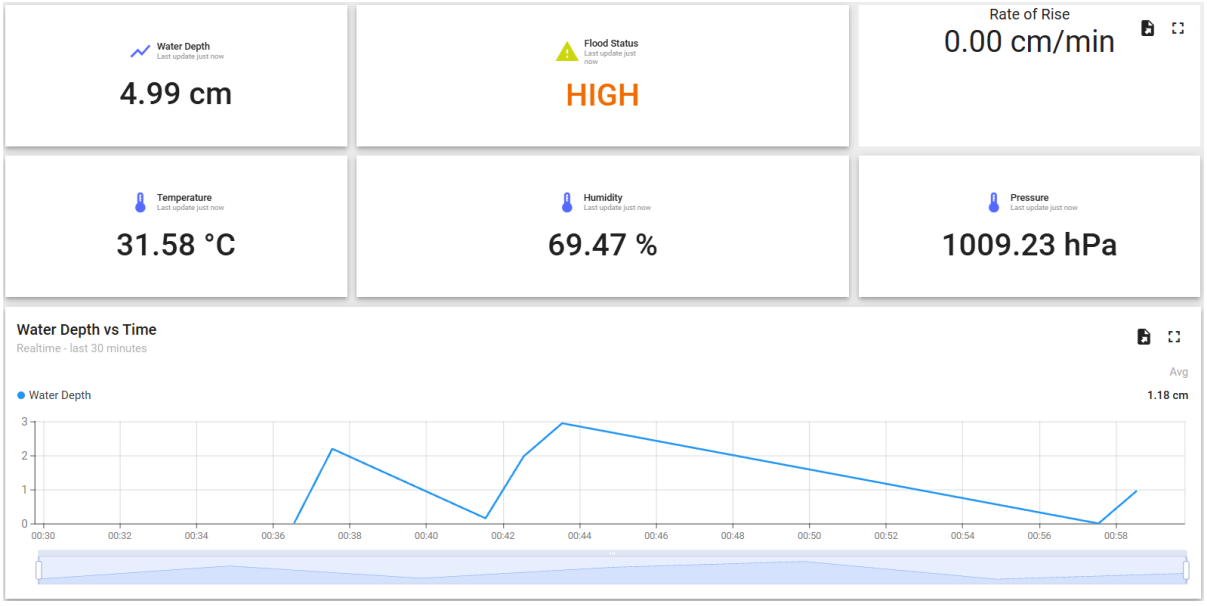


Figure 13. ThingsBoard dashboard in HIGH state: 4.99 cm water depth and 0.00 cm/min rise rate

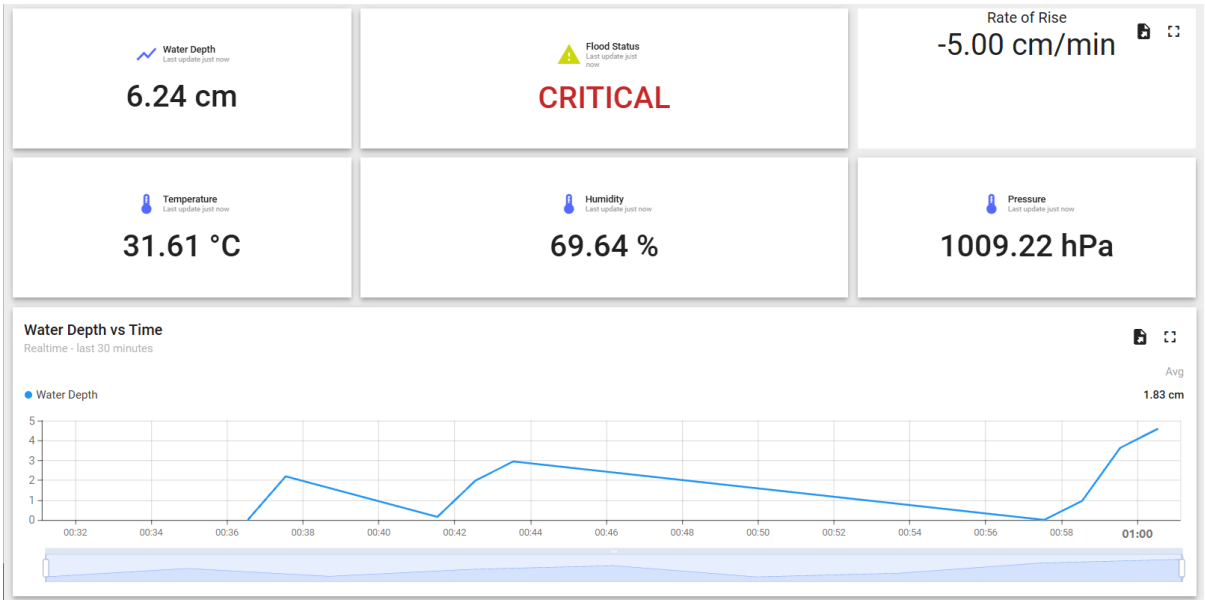


Figure 14. ThingsBoard dashboard in CRITICAL state: 6.24 cm water depth

11.4 Serial Monitor Evidence

The serial output below records the raw reference distance, measured distance, filtered distance, derived water depth, rate of rise, BME280 telemetry, and the exact JSON payload published to ThingsBoard during the Critical test.

```
------------------------------
Reference Distance : 65.87 cm
Measured Distance  : 58.69 cm
Filtered Distance  : 59.44 cm
Water Depth        : 6.43 cm
Rise Rate          : 5.00 cm/min
Status             : CRITICAL
Temperature        : 31.60 C
Humidity           : 69.54 %
Pressure           : 1009.20 hPa
ThingsBoard telemetry sent:
{"water_depth_cm":6.43,"rate_of_rise_cm_min":5.00,"temperature_C":31.60,"humidity_percent":69.54,"pressure_hPa":1009.20,"flood_status":"CRITICAL","flood_status_code":3}
------------------------------

------------------------------
Reference Distance : 65.87 cm
Measured Distance  : 59.89 cm
Filtered Distance  : 59.15 cm
Water Depth        : 6.72 cm
Rise Rate          : 3.46 cm/min
Status             : CRITICAL
Temperature        : 31.61 C
Humidity           : 69.60 %
Pressure           : 1009.23 hPa
ThingsBoard telemetry sent:
{"water_depth_cm":6.72,"rate_of_rise_cm_min":3.46,"temperature_C":31.61,"humidity_percent":69.60,"pressure_hPa":1009.23,"flood_status":"CRITICAL","flood_status_code":3}
------------------------------

------------------------------
Reference Distance : 65.87 cm
Measured Distance  : 59.87 cm
Filtered Distance  : 59.63 cm
Water Depth        : 6.24 cm
Rise Rate          : -5.00 cm/min
Status             : CRITICAL
Temperature        : 31.61 C
Humidity           : 69.64 %
Pressure           : 1009.22 hPa
ThingsBoard telemetry sent:
{"water_depth_cm":6.24,"rate_of_rise_cm_min":-5.00,"temperature_C":31.61,"humidity_percent":69.64,"pressure_hPa":1009.22,"flood_status":"CRITICAL","flood_status_code":3}
------------------------------
```

Figure 15. Serial Monitor evidence during the final Critical-level validation test

12 Advantages

- Non-contact water-level measurement using a waterproof probe.
- Real-time edge calculation of water depth and rate of rise.

- Local and remote indication through OLED/LED/buzzer plus cloud dashboard.
- Clear threshold-based status classification that is easy to interpret and demonstrate.
- Environmental telemetry through the BME280 provides additional context.
- MQTT-based cloud communication is lightweight and suitable for IoT telemetry.
- Modular architecture allows future addition of rain, flow, long-range communications, or multiple nodes.
- Open-source-oriented documentation supports hardware and software reproducibility.

13 Limitations

- The present prototype is a single monitoring node and therefore does not provide spatial coverage across multiple locations.
- Wi-Fi availability is required for real-time cloud telemetry; the local OLED/LED/buzzer continues to provide local indication when the dashboard is not viewed.
- The current startup calibration assumes a valid reference/empty condition; an already-flooded installation could produce an incorrect zero if calibration is performed at the wrong time.
- JSN-SR04T versions have different stated ranges: v2.0 is commonly specified as 25–450 cm and v3.0 as 20–600 cm [3]. The exact module version in this prototype was not recorded. Its firmware accepts readings from 22 to 600 cm; this is a configured validity window, not a verified operating range. Confirm the module version and validate its limits before deployment.
- The yellow and red LEDs use D3/GPIO0 and D4/GPIO2, which are ESP8266 boot-configuration pins. Cold-start behavior with the LED circuits attached should be checked to confirm reliable flash boot; this reset behavior was not established by the water-level tests.
- The breadboard implementation is a laboratory prototype and is not mechanically or environmentally hardened for permanent outdoor deployment.
- The current MQTT/TLS development setup used certificate-validation bypass on the ESP8266; CA-based certificate validation should be enabled for production.
- No absolute accuracy claim is made because the final validation did not use a traceable reference distance instrument; the tests verified state transitions, trend behavior, and end-to-end telemetry.

14 Future Scope

- Migrate the reference calibration to a fixed/stored installation parameter or a controlled commissioning mode.
- Enable full CA certificate validation for the MQTT/TLS connection.
- Design a custom PCB and weather-resistant enclosure for outdoor use.
- Add rainfall measurement to correlate precipitation with water-level trends.
- Add a flow/velocity sensor where site conditions permit, enabling richer hydrological analysis.
- Extend the architecture to multiple nodes using LoRa/LoRaWAN or another long-range link.
- Add solar-assisted power and battery management for remote deployments.
- Introduce SMS, mobile-app, email, or other alert channels in addition to the dashboard.
- Use more advanced filtering or anomaly detection for difficult acoustic environments.
- Integrate GIS/map views for multi-location flood monitoring and site comparison.
- Build a larger experimental dataset to evaluate noise, repeatability, response time, and field performance quantitatively.

15 Conclusion

This project successfully demonstrates an IoT-based single-station flood monitoring and early-warning prototype using a NodeMCU ESP8266, JSN-SR04T waterproof ultrasonic sensor, BME280 environmental sensor, local indicators, MQTT communication, and a ThingsBoard Cloud dashboard. The prototype performs the complete sensing-to-cloud chain: it acquires water-level data, filters the measurement, converts it to water depth, calculates a rate-of-rise indicator, classifies the water condition into Normal, Moderate, High, or Critical states, and transmits the telemetry to a remote dashboard.

The final controlled validation showed the intended state transitions at the configured thresholds. Representative observations included 0.00 cm for the empty condition, 2.11-2.54 cm in the Moderate range, 4.60-4.99 cm in the High range, and 6.24-6.72 cm in the Critical range. The dashboard mirrored the live water depth, rise rate, flood state, environmental measurements, and historical trend. The testing therefore supports the use of the prototype as an educational and engineering demonstrator for real-time flood-level monitoring.

The project also establishes a clear foundation for future development. With a hardened enclosure, fixed reference calibration, validated TLS certificates, additional hydrological sensors, multiple stations, and long-range communications, the prototype can be extended toward a more complete distributed flood-monitoring platform. The present report documents the working prototype honestly within its laboratory scope and identifies the engineering steps required for field-ready deployment.